

A Pipelined RISC-V RV32I Core on the Nexys4 DDR FPGA

A 5-stage pipeline with forwarding, hazard handling,
memory-mapped UART and GPIO peripherals

Matteo Magnaguagno

Single-author project

Advanced Logic Design
Prof. Roberto Passerone
University of Trento

Contents

1	Introduction	2
2	Overall architecture	2
3	Instruction set and encoding	3
4	Module-by-module design	3
4.1	Instruction memory (<code>instr_memory</code>)	3
4.2	Decoder (<code>decoder</code>)	3
4.3	Immediate generator (<code>immediate_gen</code>)	3
4.4	Register file (<code>regfile_dp</code>)	3
4.5	ALU (<code>alu</code>)	4
4.6	Forwarding unit (<code>forwarding_unit</code>)	4
4.7	Hazard unit (<code>hazard_unit</code>)	4
4.8	Memory-mapped bus (<code>memory_map</code>)	4
4.9	Data memory (<code>data_memory</code>)	4
4.10	UART transmitter (<code>uart_tx</code>)	4
4.11	GPIO (<code>gpio</code>)	4
4.12	Board wrapper (<code>nexys4ddr_top</code>)	4
5	Control and hazard handling	5
6	Pipeline structure and the memory map	5
7	Simulations	7
8	Implementation and performance	8
8.1	Resource utilisation	8
8.2	Timing and critical path	8
8.3	Power	9
8.4	Throughput and latency	9
9	Conclusions	9

1 Introduction

This report describes the design, implementation and validation of a **pipelined RISC-V processor** implementing a subset of the RV32I base integer instruction set, realised in VHDL and deployed on a Digilent **Nexys4 DDR** board (Xilinx Artix-7 xc7a100tcsg324-1). The work was carried out as a single-author project for the Advanced Logic Design course.

The objectives, following the project guidelines, were a working processor with a simple 4/5-stage pipeline using BRAM as instruction and data memory, and at least one peripheral so that the CPU's behaviour can be observed. The design goes beyond the minimum ISA layer by adding a **5-stage pipeline** with data-hazard **forwarding**, **load-use stall** logic and **branch handling**, plus two memory-mapped peripherals: a **UART transmitter** (to print strings to a host terminal) and a **GPIO** block (LEDs and slide switches). The instruction set was also extended with **LUI**, needed to build 32-bit peripheral addresses without a memory table.

Scope of this report. The deliverable design is the pipelined core (`cpu_top_pipelined`). An earlier multi-cycle core (`cpu_top` + `control_fsm`) was built first as a stepping stone and is *not* documented here; the modules that belong only to it (`control_fsm`, `regfile`, `pc_unit`, `comparator`) are noted as legacy where relevant.

2 Overall architecture

The processor is organised as a classic five-stage pipeline (IF → ID → EX → MEM → WB) that talks to instruction memory on the fetch side and, through a memory-mapped bus, to data memory and the two peripherals on the memory side. Figure 1 shows the top-level blocks and how they interact.

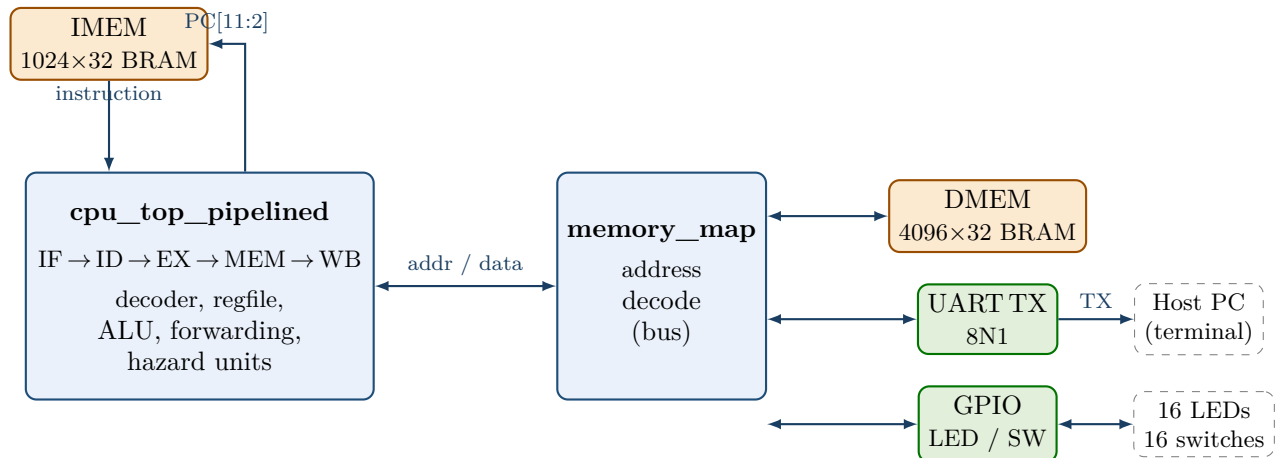


Figure 1: Top-level block diagram. The pipelined core fetches from IMEM and reaches DMEM, UART and GPIO through the `memory_map` bus, which decodes a few address bits to route reads and writes. Read latency is uniform (1 cycle) across DMEM and peripherals so the pipeline sees them identically.

How the blocks interact. Each cycle the IF stage presents `PC[11:2]` to the synchronous IMEM BRAM, whose registered output is the fetched instruction. ID decodes it (combinationally) and reads two register operands; EX computes the ALU result and resolves branches; MEM drives the `memory_map` bus with the (registered) address; WB writes back to the register file. The bus decodes `addr(16)`, `addr(3)` and `addr(2)` to select DMEM, UART or GPIO (Section 4.8).

3 Instruction set and encoding

The decoder (`decoder.vhd`) recognises the instructions listed in Table 1; every other opcode is treated as a NOP. All formats are standard RV32I.

Type	Instructions	Opcode	Notes
R	ADD, SUB, XOR, OR, AND	0110011	rd = rs1 op rs2
I	ADDI, XORI, ORI, ANDI	0010011	rd = rs1 op imm
I	LW	0000011	load word
S	SW	0100011	store word
B	BEQ, BNE, BLT, BGE	1100011	signed compare, resolved in EX
J	JAL	1101111	rd = PC+4, jump
U	LUI	0110111	rd = imm[31:12] «12

Table 1: Implemented instruction subset. Counting each mnemonic separately gives **17 instructions** (16 before the LUI extension). Excluded from the subset: shifts, SLT/SLTU, AUIPC, JALR, sub-word load/store and the M-extension.

The subset therefore totals **17 instructions**. The core originally implemented 16; LUI was added later (to build 32-bit peripheral addresses), which is why some earlier notes still quote the pre-extension count.

Immediates are produced by `immediate_gen.vhd`, which sign-extends the I/S/B/J formats and zero-fills the low 12 bits of the U format. Because the subset has no AUIPC/JALR, 32-bit addresses (e.g. peripheral base addresses) are built with the LUI + ADDI pair.

4 Module-by-module design

This section zooms into each module instantiated by the pipelined core. The hierarchy is: `cpu_top_pipelined` instantiates `instr_memory`, `decoder`, `immediate_gen`, `regfile_dp`, `hazard_unit`, `forwarding_unit`, `alu` and `memory_map`; the latter wraps `data_memory`, `uart_tx` and `gpio`.

4.1 Instruction memory (`instr_memory`)

A synchronous **1024×32** BRAM addressed by PC[11:2] (word level). The read is registered, so the instruction appears one cycle after the address; in the pipeline this registered output *is* the IF/ID register (Section 6). A read-enable input freezes the output during a stall. Programs are selected at elaboration time through the `PROGRAM_SEL` generic; the test/demo programs are listed in Table 3.

4.2 Decoder (`decoder`)

Purely combinational. From the opcode/funct fields it emits a one-hot `op_class` (ALU / Store / Load / Branch / Jump), the 3-bit ALU opcode, the branch condition code, the operand-select signals `a_sel` (PC vs. rs1) and `b_sel` (rs2 vs. immediate), the immediate-format selector, and an `is_lui` flag. Defaults are assigned at the top of the process to avoid inferred latches.

4.3 Immediate generator (`immediate_gen`)

Combinational extraction and sign-extension of the I, S, B, U and J immediate formats from the instruction word.

4.4 Register file (`regfile_dp`)

A dual-port **32×32** register file: two *asynchronous* read ports (synthesised as LUT/distributed RAM) and one *synchronous* write port. `x0` is hard-wired to zero. It implements a **write-first**

bypass: when a register is written in WB and read in ID in the same cycle, the read returns the new value. This closes the WB→ID hazard without an extra forwarding path.

4.5 ALU (**alu**)

32-bit, operations ADD, SUB, XOR, OR, AND (3-bit opcode). It exposes both a combinational **alu_pre_result** (used to address DMEM one cycle early, hiding the BRAM latency) and a registered **alu_result**.

4.6 Forwarding unit (**forwarding_unit**)

Combinational. For each EX operand it selects between the register-file value, the EX/MEM result, or the MEM/WB result, giving priority to the more recent EX/MEM stage. It never forwards from/to x0. This resolves ordinary RAW hazards without stalling.

4.7 Hazard unit (**hazard_unit**)

Combinational. Detects the **load-use** hazard — a LW in EX whose destination is read by the instruction in ID — and raises **stall**, freezing PC and IF/ID for one cycle and injecting a bubble into ID/EX. This is the one case forwarding alone cannot cover, because the loaded datum is not ready until the end of MEM.

4.8 Memory-mapped bus (**memory_map**)

Routes a 32-bit address to DMEM or to a peripheral by decoding three bits (Table 2). Writes are combinational on the current address; reads are given a **uniform 1-cycle latency** (the peripheral select and value are registered) so that DMEM and peripheral reads land in WB at the same time the pipeline expects them.

4.9 Data memory (**data_memory**)

Synchronous **4096×32** BRAM (16 kB), word-addressed by **alu_result[13:2]**. Initialised with peripheral base addresses and the “Hello” ASCII string for the legacy demo; the pipelined demos build their data at run time instead.

4.10 UART transmitter (**uart_tx**)

An 8N1 transmitter driven by a 4-state FSM (IDLE → START → DATA → STOP). The bit period is $\text{BAUD_DIV} = \text{CLK_HZ}/\text{BAUD} = 868$ cycles at 100 MHz / 115200 baud. There is no transmit buffer: **tx_busy** is high for the whole frame and is exposed (inverted) as UART_STATUS bit 0 (“ready”), so software must poll before sending each byte.

4.11 GPIO (**gpio**)

A 16-bit output register driving the LEDs, and a 16-bit switch input passed through a **two-flip-flop synchroniser** to avoid metastability on the asynchronous switch lines.

4.12 Board wrapper (**nexys4ddr_top**)

A thin top level that adapts the board to the core: it inverts the active-low CPU_RESET button, fixes PROGRAM_SEL, and leaves the debug ports **open** so synthesis prunes them. The XDC constraints bind **clk**, **cpu_resetsn**, **uart_tx_pin**, **led_out** and **sw_in** to physical pins.

5 Control and hazard handling

Unlike the legacy multi-cycle core, the pipeline has **no control FSM**. Control is fully distributed: the combinational **decoder** produces the control signals, which then travel down the pipeline registers (ID/EX → EX/MEM → MEM/WB) alongside the data; the **hazard_unit** and **forwarding_unit** are combinational; and branches are resolved in EX. There is therefore no state diagram for the datapath — the right way to show the control behaviour is the pipeline timing diagram of Figure 2, which exposes the three mechanisms at work.

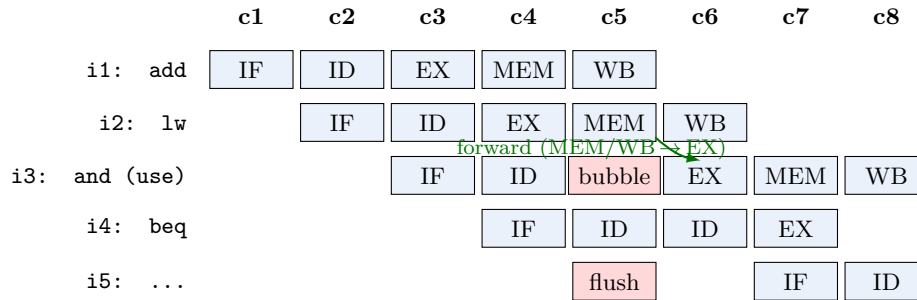


Figure 2: Pipeline timing. i3 uses the value loaded by i2 (load-use): the hazard unit inserts a one-cycle **bubble** and the value is then **forwarded** from MEM/WB. A taken branch at i4 (resolved in EX) **flushes** the two younger instructions. The same events are shown on a real simulation trace in Figure 3.

Branch / control hazard. Branches and JAL are resolved in EX; on a taken branch the PC is redirected and the two instructions already fetched are squashed (a valid bit forces a NOP into ID, and ID/EX is cleared), for a 2-cycle penalty. No branch prediction is used.

6 Pipeline structure and the memory map

The pipeline keeps the canonical four pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB). The key design decision concerns the BRAMs.

BRAM output register as pipeline register. IMEM and DMEM are synchronous-read BRAMs: their output is already registered (one cycle of latency). Rather than adding a *second* register downstream — which would create hidden stages and misalignment bugs — the BRAM output register is used *as* the pipeline register: the IMEM output is the IF/ID instruction, and the DMEM output is the MEM/WB load datum, read live in WB. Only what the BRAM does not carry (the companion PC and a valid bit) is registered separately.

```

1 imem_re <= not stall;           -- freeze IMEM output during stall
2 u_imem: entity work.instr_memory
3     generic map ( PROGRAM_SEL => PROGRAM_SEL )
4     port map ( clk => clk, re => imem_re,
5               addr => pc_word_if, instruction => instruction_if );
6 -- squash on flush/reset: the decoder sees a NOP
7 id_instr <= instruction_if when ifid_valid = '1' else NOP_INSTR;

```

Listing 1: IF stage: the IMEM output register is the IF/ID instruction; a read-enable freezes it during a stall.

LUI handled in EX. LUI reuses the ALU adder: the decoder raises `is_lui` and in EX operand A is forced to zero, so `rd = 0 + imm[31:12]◀12`. Doing this in EX (not ID) keeps the register-read path off the critical path.

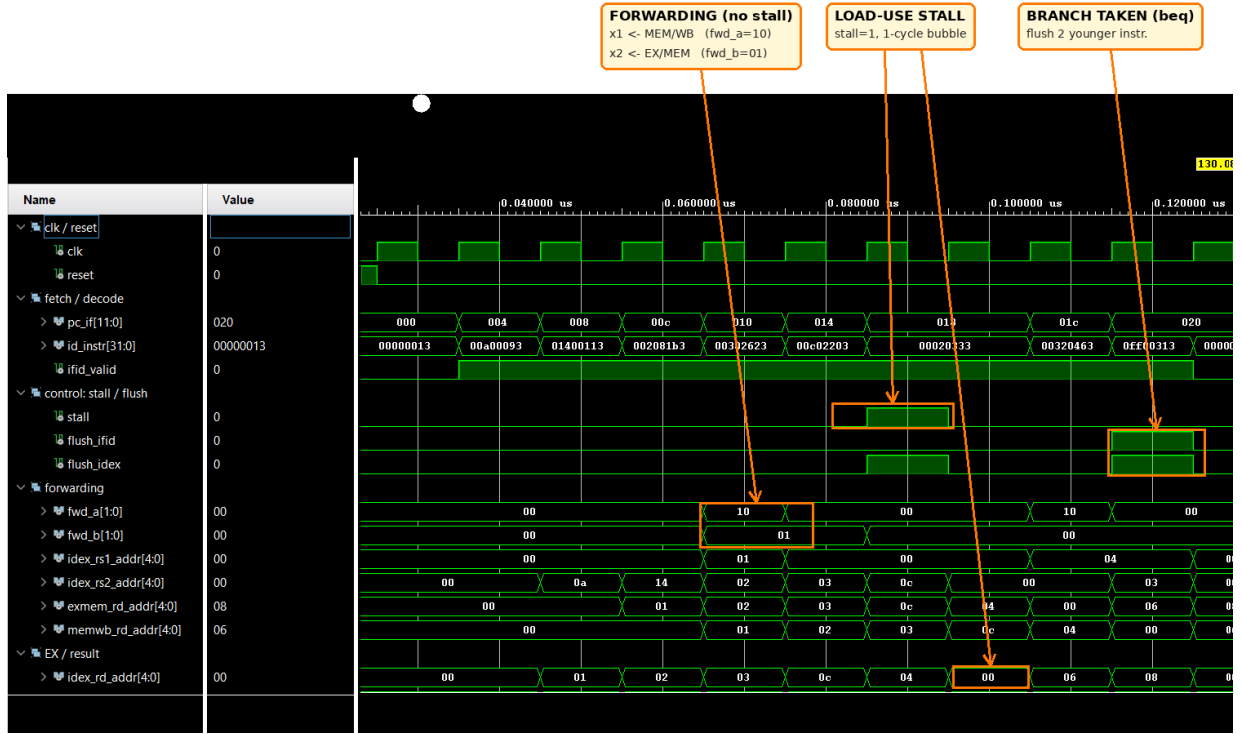


Figure 3: Annotated simulation waveform of PROGRAM_E, showing the three control mechanisms in one trace. **Forwarding** (no stall): `add x3,x1,x2` receives `x1` from MEM/WB (`fwd_a=10`) and `x2` from EX/MEM (`fwd_b=01`). **Load-use stall**: `lw x4` followed by `add x6,x4` raises `stall=1` for one cycle and injects a bubble (`idex_rd=0`); `x4` is then forwarded from MEM/WB. **Branch flush**: the taken `beq` (resolved in EX) squashes the two younger instructions (`flush_ifid/flush_idx=1`). The run ends with the `led_out=0x4F` signature asserted by the testbench.

```

1 alu_a_ex <= (others => '0') when idex_is_lui = '1' -- LUI: rd = 0 + immU
2   else pc_ex_32      when idex_a_sel = '0'      -- branch/JAL: PC
3   else rs1_fwd;      -- normal: rs1 (forwarded)

```

Listing 2: EX stage: operand-A mux, with the LUI zero-forcing.

Memory map. Table 2 gives the address map. The bus decodes `addr(16)` (DMEM vs. peripheral), `addr(3)` (UART vs. GPIO) and `addr(2)` (data vs. status register).

Region	Address	Access	Notes
DMEM (16 kB)	0x0000_0000–0x0000_3FFF	R/W	data memory
UART_DATA	0x0001_0000	W	write a byte to transmit
UART_STATUS	0x0001_0004	R	bit 0 = ready (not busy)
GPIO_LED	0x0001_0008	W	16 LEDs
GPIO_SW	0x0001_000C	R	16 switches (synchronised)

Table 2: Memory-mapped address space.

UART frame. Each byte is sent as an 8N1 frame: a start bit (line low), 8 data bits LSB-first, and a stop bit (line high), each lasting `BAUD_DIV` clock cycles (Figure 4).

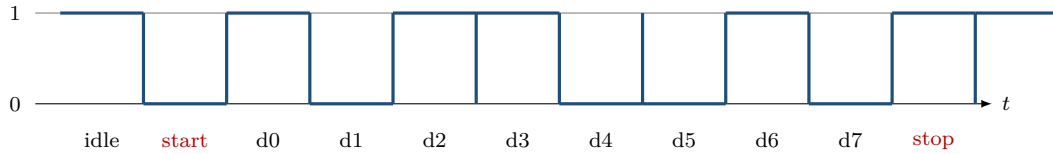


Figure 4: UART 8N1 frame (example byte). Line idles high; the falling edge of the start bit marks the beginning of the frame; data are sent LSB first.

Test and demo programs. Programs are pre-loaded in `instr_memory.vhd` and chosen with `PROGRAM_SEL` (Table 3).

SEL	Name	Purpose
4	PROGRAM_E	self-checking hazard test (forwarding, stall, branch); LED signature 0x4F
5	PROGRAM_F	switch → LED echo (peripheral read in the pipeline)
7	PROGRAM_H	“Hello” over UART with polling (the main demo)
8	PROGRAM_I	<i>running light</i> on the LEDs (ALU, branch, delay loop)
9	PROGRAM_J	interactive: switch echo to LEDs, switch 15 sends ‘A’ over UART

Table 3: Selected programs used for validation and the live demo (SEL 8 and 9 were added for this report).

7 Simulations

The design was validated by behavioural simulation of the testbenches and by running the demo programs. The following figures show the important aspects of the design; capture them from the simulator (Vivado xsim or GHDL) on the indicated program.

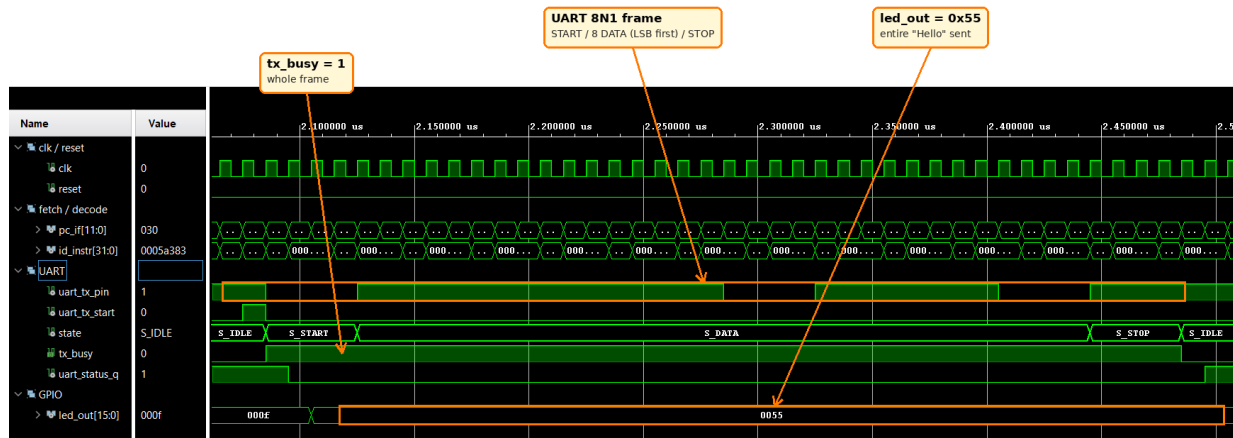


Figure 5: Annotated behavioural simulation of PROGRAM_H (Hello over UART). For each character the program polls `UART_STATUS` until ready and writes `UART_DATA`, producing an 8N1 frame on `uart_tx_pin` (start / 8 data bits, LSB first / stop), with `tx_busy` high for the whole frame. The frame shown is the last character (‘o’); `led_out` is already 0x0055, the end-of-program signature, because the CPU kicked the final byte and moved on while the UART, being asynchronous, finishes transmitting. A fast baud (`BAUD_DIV=4`) is used in simulation so frames are short; the board keeps 115200.

The hazard behaviour (forwarding, load-use stall and branch flush) is shown in detail by the annotated waveform of PROGRAM_E in Figure 3 (Section 5); it is not repeated here.

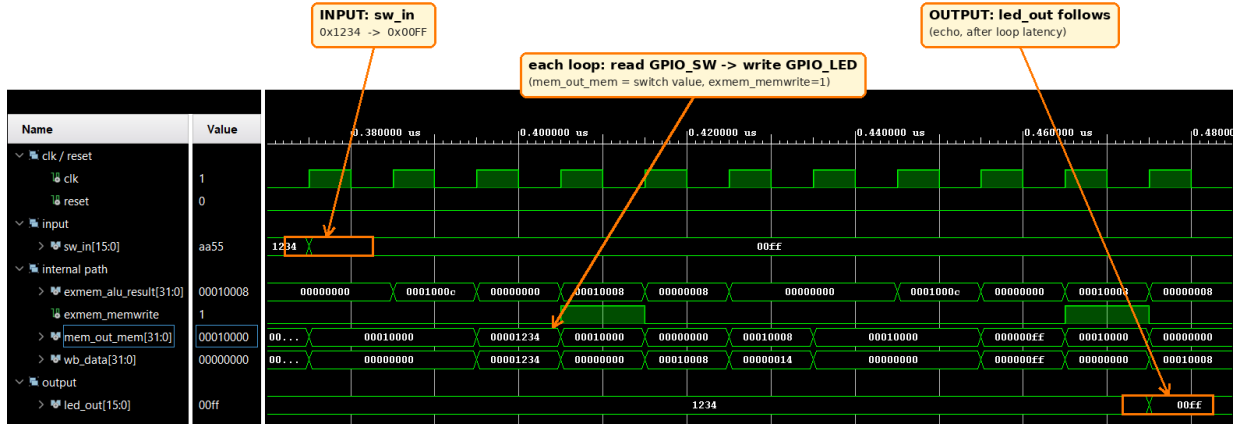


Figure 6: Annotated behavioural simulation of PROGRAM_F (switch → LED echo). The program loops reading GPIO_SW (mem_out_mem) and writing GPIO_LED (exmem_memwrite=1 at address 0x10008). When sw_in changes (0x1234→0x00FF), led_out follows one loop iteration later — the lag is the memory-mapped read plus the loop/pipeline latency, exactly the real-time echo seen on the board.

8 Implementation and performance

The design was synthesised and implemented in Vivado targeting the Artix-7 xc7a100tcsg324-1 at a 100 MHz system clock. Timing closure used the **Performance_Retiming** implementation strategy.

8.1 Resource utilisation

The implemented design is tiny relative to the device (Table 4): **611 LUTs** (0.96%) and **401 flip-flops** (0.32%). Of the LUTs, 48 are used as distributed RAM (the dual-port register file regfile_dp) and 563 as logic. The design uses **5 Block RAM tiles** (RAMB36E1, 3.7%) and **no DSP** (the ALU needs no multiplier). The five BRAMs split as about four tiles for the 16 kB data memory and one for the 4 kB instruction memory, so the **data memory is the most BRAM-demanding block**; the LUT/FF budget is dominated by the pipeline core.

Resource	Used	Available	%
Slice LUTs	611	63400	0.96%
Slice Registers (FF)	401	126800	0.32%
Block RAM (RAMB36)	5	135	3.70%
DSP	0	240	0.00%

Table 4: Post-implementation resource utilisation (**Performance_Retiming** run).

8.2 Timing and critical path

At the 100 MHz target (clock period $T = 10$ ns) the implemented design **meets all timing constraints**: 0 failing endpoints out of 1312, with worst setup slack **WNS**= +0.053 ns, worst hold slack **WHS**= +0.057 ns (both positive) and worst pulse-width slack +3.750 ns. The small but positive setup margin gives a maximum frequency

$$F_{max} = \frac{1}{T - \text{WNS}} = \frac{1}{9.947 \text{ ns}} \approx 100.5 \text{ MHz},$$

i.e. the design closes 100 MHz essentially at the limit with the **Performance_Retiming** strategy.

The worst setup path (slack +0.053 ns) runs from the **DMEM BRAM read-data output register** (`u_cpu/u_mmap/u_dmem/mem_reg`) to the **address input of a Block RAM** (`ADDRARDADDR`): a loaded value flows through the register-file write-first bypass into an ALU operand and on to the next memory-address computation — exactly the feedback loop the pipeline was designed around. The data-path delay is **9.305 ns** of the 10 ns budget, split **4.243 ns logic (45.6%) / 5.062 ns route (54.4%)** over **9 logic levels** (3 carry chains + 6 LUTs). Routing slightly dominates, which is why the `Performance_Retiming` strategy — rebalancing registers along this path — is what pulls the slack positive.

Metric	Value
Target clock period	10 ns (100 MHz)
WNS (setup)	+0.053 ns
WHS (hold)	+0.057 ns
F_{max}	≈ 100.5 MHz
Critical path delay	9.305 ns (4.243 logic / 5.062 route)

Table 5: Timing summary (final `Performance_Retiming` run).

8.3 Power

The post-implementation estimate is a **total on-chip power of 0.129 W**, split into **0.032 W dynamic** and **0.097 W device static**. The static (leakage) component dominates, as expected for such a small, lightly-switching design. The estimated junction temperature is 25.6 °C (effective $\theta_{JA} = 4.6$ °C/W, max ambient 84.4 °C), so the design runs thermally cold. Vivado flags a *Low* confidence level because no switching-activity file was provided, so this is a vectorless estimate.

8.4 Throughput and latency

With a one-instruction-per-cycle pipeline at the closed clock frequency, the ideal throughput is ≈ 100.5 M instructions/s (one per cycle), minus the penalties: **1 cycle** per load-use hazard and **2 cycles** per taken branch.

9 Conclusions

The project delivers a working pipelined RISC-V RV32I-subset processor on the Nexys4 DDR, with forwarding, load-use stalling and branch handling, two memory-mapped peripherals (UART and GPIO) and a small set of demonstration programs. It meets timing at 100 MHz and its behaviour is observable both on a host terminal (UART) and on the board’s LEDs and switches (GPIO).

Limitations and possible extensions. The subset omits shifts, set-less-than, AUIPC/JALR, sub-word memory access and the M-extension; there is no branch prediction (branches always pay the 2-cycle penalty). Natural extensions, in line with the guidelines, would be a multiply/-CORDIC functional unit with an ISA extension, sub-word load/store, or a 1-bit branch predictor.

Note on sources

All technical claims in this report were checked against the VHDL sources under `src/`; the resource, timing and power figures come from the Vivado implementation reports (`Performance_Retiming` run) and the simulation waveforms from the project testbenches.